Exceptions in Java

Last Updated : 10 Sep, 2024

Exception Handling in Java is one of the effective means to handle runtime errors so that the regular flow of the application can be preserved. Java Exception Handling is a mechanism to handle runtime errors such as ClassNotFoundException, IOException, SQLException, RemoteException, etc.

What are Java Exceptions?

In Java, Exception is an unwanted or unexpected event, which occurs during the execution of a program, i.e. at run time, that disrupts the normal flow of the program's instructions. Exceptions can be caught and handled by the program. When an exception occurs within a method, it creates an object. This object is called the exception object. It contains information about the exception, such as the name and description of the exception and the state of the program when the exception occurred.

Major reasons why an exception Occurs

- Invalid user input
- Device failure
- Loss of network connection
- Physical limitations (out-of-disk memory)
- Code errors
- Out of bound
- Null reference
- Type mismatch
- Opening an unavailable file
- Database errors
- Arithmetic errors

Errors represent irrecoverable conditions such as Java virtual machine (JVM) running out of memory, memory leaks, stack overflow errors, library

incompatibility, infinite recursion, etc. Errors are usually beyond the control of the programmer, and we should not try to handle errors.

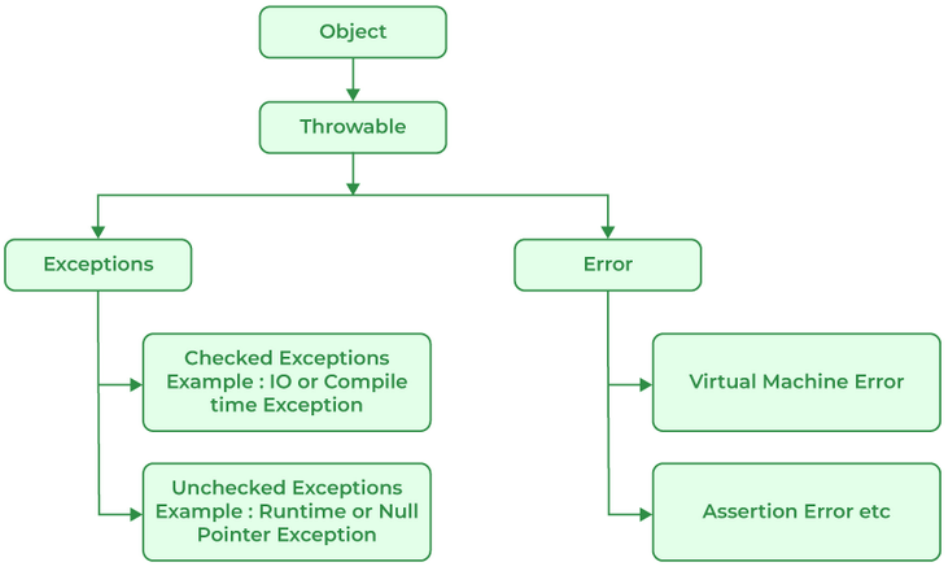
Difference between Error and Exception

Let us discuss the most important part which is the **differences between Error and Exception** that is as follows:

- **Error:** An Error indicates a serious problem that a reasonable application should not try to catch.
- **Exception:** Exception indicates conditions that a reasonable application might try to catch.

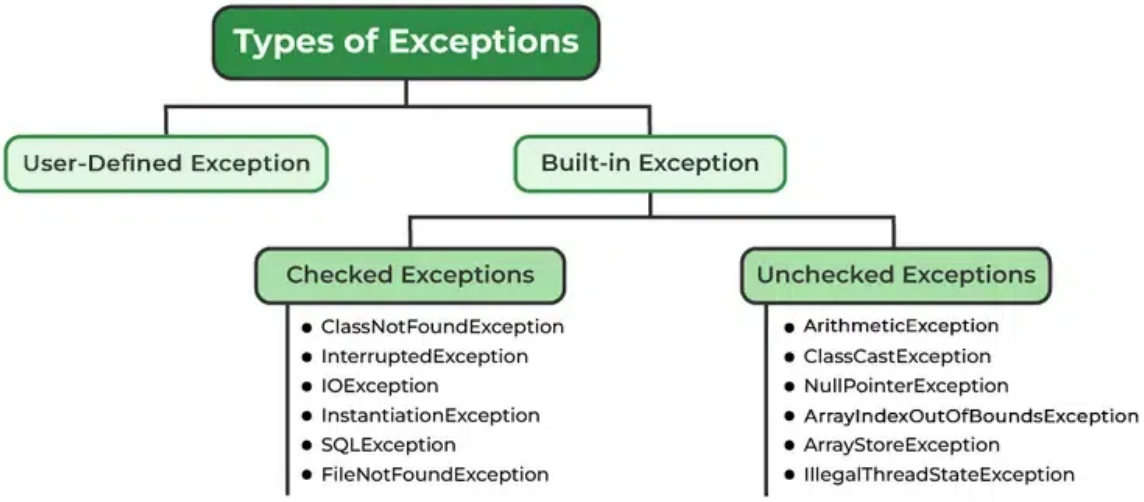
Exception Hierarchy

All exception and error types are subclasses of the class **Throwable**, which is the base class of the hierarchy. One branch is headed by **Exception**. This class is used for exceptional conditions that user programs should catch. NullPointerException is an example of such an exception. Another branch, **Error** is used by the Java run-time system(JVM) to indicate errors having to do with the run-time environment itself(JRE). StackOverflowError is an example of such an error.



Types of Exceptions

Java defines several types of exceptions that relate to its various class libraries. Java also allows users to define their own exceptions.



Exceptions can be categorized in two ways:

1. **Built-in Exceptions**
   - Checked Exception
   - Unchecked Exception

2. **User-Defined Exceptions**

Let us discuss the above-defined listed exception that is as follows:

1. Built-in Exceptions

Built-in exceptions are the exceptions that are available in Java libraries. These exceptions are suitable to explain certain error situations.

- **Checked Exceptions:** Checked exceptions are called compile-time exceptions because these exceptions are checked at compile-time by the compiler.

- **Unchecked Exceptions:** The unchecked exceptions are just opposite to the checked exceptions. The compiler will not check these exceptions at compile time. In simple words, if a program throws an unchecked exception, and even if we didn't handle or declare it, the program would not give a compilation error.

> ***Note:*** *For checked vs unchecked exception, see Checked vs Unchecked Exceptions*

2. User-Defined Exceptions:

Sometimes, the built-in exceptions in Java are not able to describe a certain situation. In such cases, users can also create exceptions, which are called 'user-defined Exceptions'.

The ***advantages of Exception Handling in Java*** are as follows:

1. Provision to Complete Program Execution
2. Easy Identification of Program Code and Error-Handling Code
3. Propagation of Errors
4. Meaningful Error Reporting
5. Identifying Error Types

Methods to print the Exception information:

1. printStackTrace()

This method prints exception information in the format of the Name of the exception: description of the exception, stack trace.

Example:

Java

```java
//program to print the exception information using printStackTrace() method

import java.io.*;

class GFG {
    public static void main (String[] args) {
      int a=5;
      int b=0;
        try{
          System.out.println(a/b);
        }
      catch(ArithmeticException e){
        e.printStackTrace();
      }
    }
```

```
16  }
```

Output

```
java.lang.ArithmeticException: / by zero
at GFG.main(File.java:10)
```

2. toString()

The toString() method prints exception information in the format of the Name of the exception: description of the exception.

Example:

Java

```java
//program to print the exception information using
toString() method

import java.io.*;

class GFG1 {
    public static void main (String[] args) {
      int a=5;
      int b=0;
        try{
          System.out.println(a/b);
        }
      catch(ArithmeticException e){
        System.out.println(e.toString());
      }
    }
}
```

Output

```
java.lang.ArithmeticException: / by zero
```

3. getMessage()

The getMessage() method prints only the description of the exception.

Example:

Java

```java
//program to print the exception information using getMessage() method

import java.io.*;

class GFG1 {
    public static void main (String[] args) {
      int a=5;
      int b=0;
        try{
          System.out.println(a/b);
        }
      catch(ArithmeticException e){
        System.out.println(e.getMessage());
      }
    }
}
```

Output

```
/ by zero
```

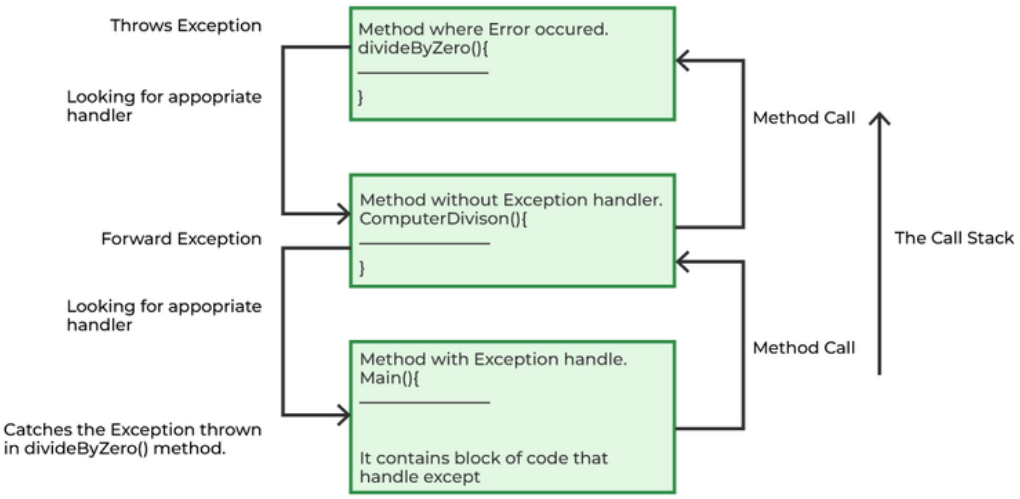
How Does JVM Handle an Exception?

Default Exception Handling: Whenever inside a method, if an exception has occurred, the method creates an Object known as an Exception Object and hands it off to the run-time system(JVM). The exception object contains the name and description of the exception and the current state of the program where the exception has occurred. Creating the Exception Object and handling it in the run-time system is called throwing an Exception. There might be a list of the methods that had been called to get to the method where an exception occurred. This ordered list of methods is called **Call Stack**. Now the following procedure will happen.

- The run-time system searches the call stack to find the method that contains a block of code that can handle the occurred exception. The block of the code is called an **Exception handler**.
- The run-time system starts searching from the method in which the exception occurred and proceeds through the call stack in the reverse order in which methods were called.
- If it finds an appropriate handler, then it passes the occurred exception to it. An appropriate handler means the type of exception object thrown matches the type of exception object it can handle.
- If the run-time system searches all the methods on the call stack and couldn't have found the appropriate handler, then the run-time system handover the Exception Object to the **default exception handler**, which is part of the run-time system. This handler prints the exception information in the following format and terminates the program **abnormally**.

```
Exception in thread "xxx" Name of Exception : Description
... ..... ..  // Call Stack
```

Look at the below diagram to understand the flow of the call stack.



The Call Stack and searching the call stack for exception handler.

Illustration:

Java

```java
// Java Program to Demonstrate How Exception Is Thrown

// Class
// ThrowsExecp
class GFG {

    // Main driver method
    public static void main(String args[])
    {
        // Taking an empty string
        String str = null;
        // Getting length of a string
        System.out.println(str.length());
    }
}
```

Output

```
mayanksolanki@MacBook-Air Desktop % javac GFG.java
mayanksolanki@MacBook-Air Desktop % java GFG
Exception in thread "main" java.lang.NullPointerException: Cannot invoke
"String.length()" because "<local1>" is null
        at GFG.main(GFG.java:12)
mayanksolanki@MacBook-Air Desktop %
```

Let us see an example that illustrates how a run-time system searches for appropriate exception handling code on the call stack.

Example:

Java

```java
// Java Program to Demonstrate Exception is Thrown
// How the runTime System Searches Call-Stack
// to Find Appropriate Exception Handler

// Class
// ExceptionThrown
class GFG {

    // Method 1
    // It throws the Exception(ArithmeticException).
    // Appropriate Exception handler is not found
    // within this method.
    static int divideByZero(int a, int b)
    {

        // this statement will cause
ArithmeticException
        // (/by zero)
        int i = a / b;

        return i;
    }

    // The runTime System searches the appropriate
    // Exception handler in method also but couldn't
have
    // found. So looking forward on the call stack
```

```
26      static int computeDivision(int a, int b)
27      {
28
29          int res = 0;
30
31          // Try block to check for exceptions
32          try {
33
34              res = divideByZero(a, b);
35          }
36
37          // Catch block to handle NumberFormatException
38          // exception Doesn't matches with
39          // ArithmeticException
40          catch (NumberFormatException ex) {
41              // Display message when exception occurs
42              System.out.println(
43                  "NumberFormatException is occurred");
44          }
45          return res;
46      }
47
48      // Method 2
49      // Found appropriate Exception handler.
50      // i.e. matching catch block.
51      public static void main(String args[])
52      {
53
54          int a = 1;
55          int b = 0;
56
57          // Try block to check for exceptions
58          try {
59              int i = computeDivision(a, b);
60          }
61
62          // Catch block to handle ArithmeticException
63          // exceptions
64          catch (ArithmeticException ex) {
65
66              // getMessage() will print description
67              // of exception(here / by zero)
```

```
68                 System.out.println(ex.getMessage());
69             }
70         }
71     }
```

Output

```
/ by zero
```

How Programmer Handle an Exception?

Customized Exception Handling: Java exception handling is managed via five keywords: **try**, **catch**, throw, throws, and **finally**. Briefly, here is how they work. Program statements that you think can raise exceptions are contained within a try block. If an exception occurs within the try block, it is thrown. Your code can catch this exception (using catch block) and handle it in some rational manner. System-generated exceptions are automatically thrown by the Java run-time system. To manually throw an exception, use the keyword throw. Any exception that is thrown out of a method must be specified as such by a throws clause. Any code that absolutely must be executed after a try block completes is put in a finally block.

> ***Tip:*** *One must go through control flow in try catch finally block for better understanding.*

Need for try-catch clause(Customized Exception Handling)

Consider the below program in order to get a better understanding of the try-catch clause.

Example:

Java

```
1  // Java Program to Demonstrate
2  // Need of try-catch Clause
3
```

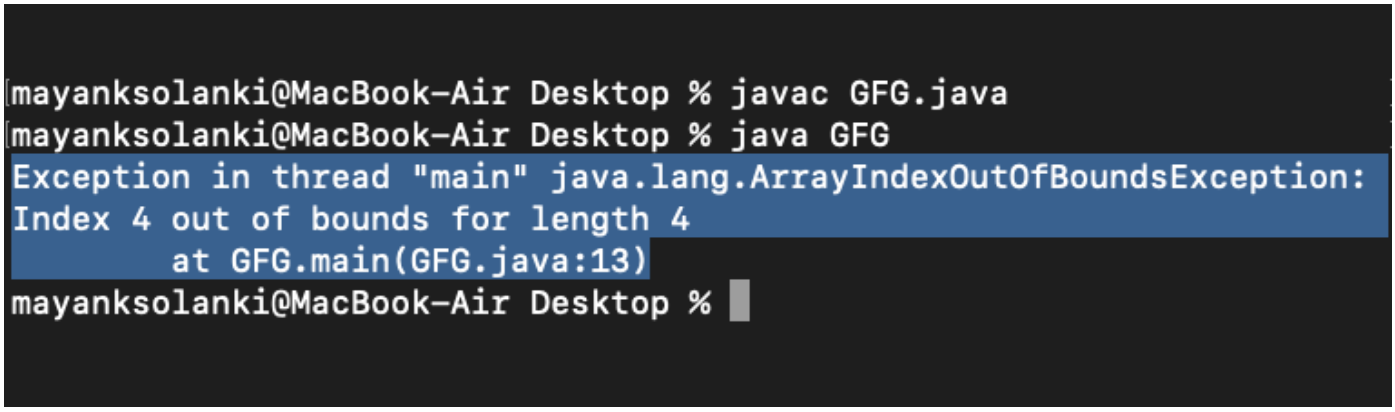
```java
// Class
class GFG {

    // Main driver method
    public static void main(String[] args)
    {
        // Taking an array of size 4
        int[] arr = new int[4];

        // Now this statement will cause an exception
        int i = arr[4];

        // This statement will never execute
        // as above we caught with an exception
        System.out.println("Hi, I want to execute");
    }
}
```

Output



Unchecked exception

Output explanation: In the above example, an array is defined with size i.e. you can access elements only from index 0 to 3. But you trying to access the elements at index 4(by mistake) that's why it is throwing an exception. In this case, JVM terminates the program **abnormally**. The statement System.out.println("Hi, I want to execute"); will never execute. To execute it, we must handle the exception using try-catch. Hence to continue the normal flow of the program, we need a try-catch clause.

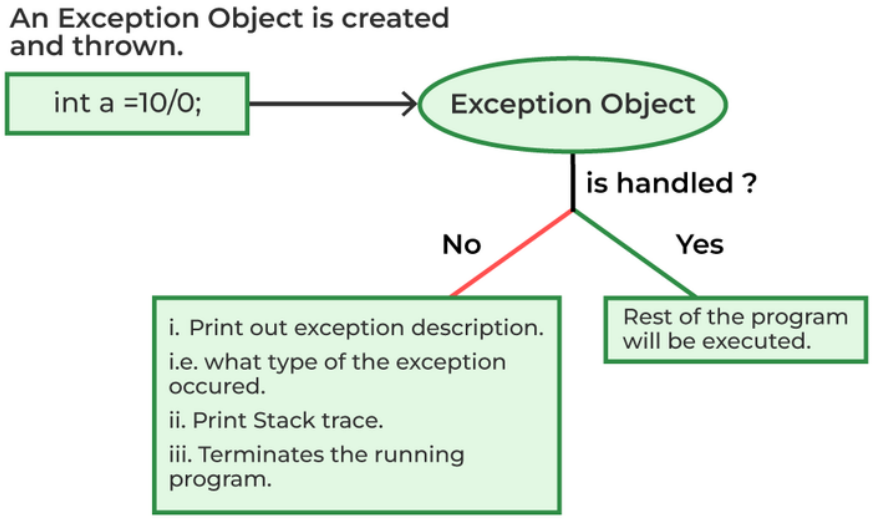
How to Use the Try-catch Clause?

```
try {
    // block of code to monitor for errors
    // the code you think can raise an exception
} catch (ExceptionType1 exOb) {
    // exception handler for ExceptionType1
} catch (ExceptionType2 exOb) {
    // exception handler for ExceptionType2
}
// optional
finally {  // block of code to be executed after try block ends
}
```

Certain key points need to be remembered that are as follows:

- In a method, there can be more than one statement that might throw an exception, So put all these statements within their own **try** block and provide a separate exception handler within their own **catch** block for each of them.
- If an exception occurs within the **try** block, that exception is handled by the exception handler associated with it. To associate the exception handler, we must put a **catch** block after it. There can be more than one exception handler. Each **catch** block is an exception handler that handles the exception to the type indicated by its argument. The argument, ExceptionType declares the type of exception that it can handle and must be the name of the class that inherits from the **Throwable** class.
- For each try block, there can be zero or more catch blocks, but **only one** final block.
- The finally block is optional. It always gets executed whether an exception occurred in try block or not. If an exception occurs, then it will be executed after **try and catch blocks.** And if an exception does not occur, then it will be executed after the **try** block. The finally block in Java is used to put important codes such as clean-up code e.g., closing the file or closing the connection.
- If we write System.exit in the try block, then finally block will not be executed.

The summary is depicted via visual aid below as follows:



Related Articles:

- Types of Exceptions in Java
- Checked vs Unchecked Exceptions
- Throw-Throws in Java

Overview of Exception Handling In Java

Visit Course ↗

Comment

More info

Next Article

Checked vs Unchecked Exceptions in Java

Similar Reads

Java Tutorial

Java is one of the most popular and widely used programming languages which is used to develop mobile apps, desktop apps, web apps, web servers, games, and enterprise-level systems. Java was...

4 min read

Java Overview

Java Basics

Java Flow Control

Java Methods

Java Arrays

Java Strings

Java OOPs Concepts

Java Interfaces

Java Collections

Java Exception Handling

Exceptions in Java

Exception Handling in Java is one of the effective means to handle runtime errors so that the regular flow of the application can be preserved. Java Exception Handling is a mechanism to handle runtime...

10 min read

Checked vs Unchecked Exceptions in Java

In Java, Exception is an unwanted or unexpected event, which occurs during the execution of a program, i.e. at run time, that disrupts the normal flow of the program's instructions. In Java, there are two types ...

5 min read

Java Try Catch Block

In Java exception is an "unwanted or unexpected event", that occurs during the execution of the program. When an exception occurs, the execution of the program gets terminated. To avoid these...

4 min read

final, finally and finalize in Java

This is an important question concerning the interview point of view. final keyword final (lowercase) is a reserved keyword in java. We can't use it as an identifier, as it is reserved. We can use this keyword wit...

13 min read

throw and throws in Java

In Java, Exception Handling is one of the effective means to handle runtime errors so that the regular flow of the application can be preserved. Java Exception Handling is a mechanism to handle runtime...

4 min read

User-defined Custom Exception in Java

An exception is an issue (run time error) that occurred during the execution of a program. When an exception occurred the program gets terminated abruptly and, the code past the line that generated th...

3 min read

Chained Exceptions in Java

Chained Exceptions in Java allow associating one exception with another, i.e. one exception describes the cause of another exception. For example, consider a situation in which a method throws an...

3 min read

Null Pointer Exception in Java

A NullPointerException in Java is a RuntimeException. In Java, a special null value can be assigned to an object reference. NullPointerException is thrown when a program attempts to use an object reference...

5 min read

Exception Handling with Method Overriding in Java

An Exception is an unwanted or unexpected event, which occurs during the execution of a program i.e at run-time, that disrupts the normal flow of the program's instructions. Exception handling is used to...

6 min read

Java Multithreading

Java File Handling

Java Streams and Lambda Expressions

Java IO

Java Synchronization

Java Regex

Java Networking

JDBC

Java Memory Allocation

Java Interview Questions

Java Practice Problems

Java Projects

Article Tags : Java School Programming Java-Exceptions

Practice Tags : Java